

同济大学计算机系

数字逻辑课程综合实验报告



学 号 2354366

姓 名 徐华炫

专 业 基础学科拔尖基地(计算机)

授课老师 张冬冬

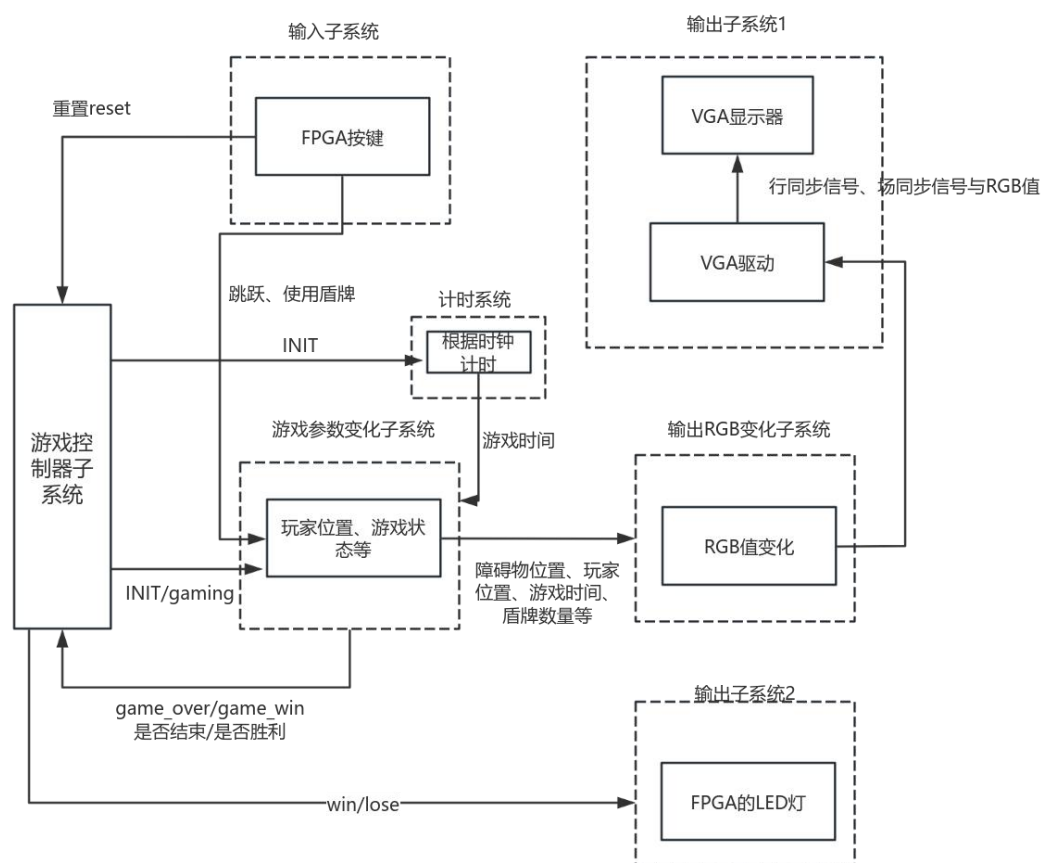
一、实验内容

基于 verilog 语言，使用 vga 显示器，在可编程 FPGA 上实现类“chrome 小恐龙”的小游戏。

游戏通过开发板上的按键控制，游戏画面显示在 vga 显示器上。游戏的基本规则是：玩家控制代表“小恐龙”的绿色方块进行跳跃，以躲避前方刷新的障碍物，撞到障碍物即游戏结束，障碍物的速度会随着游戏时间增加而变快，屏幕上方显示总游戏通关进度条，当坚持到进度条满即通关。其中障碍物分为紫色空中障碍物和红色地面障碍物，位置和高度均随机刷新；玩家有五次使用“防护盾”的技能次数，使用时玩家方块变成蓝色，能无敌通过障碍物。“防护盾”共能使用五次。

二、“小恐龙”游戏数字系统总框图

（按由顶向下方法进行子系统的划分，给出包含各子系统相互关系及控制信号的总框图，并对各子系统功能及实现进行概述。具体可参考教材 183 页的相关描述方法。）



输入子系统：FPGA 的按键 M18 和 P18，M18 为 key_up，控制玩家的跳跃；P18 为 key_down，控制玩家的技能(盾牌)使用。

输出子系统 1：用于显示游戏画面，使用的是 VGA 显示器，其需要的行同步信号、场同步信号由 VGA 驱动模块产生并输入，RGB 信号由 RGB 值控制模块产生并输入。

输出子系统 2：FPGA 的 led 灯 H17 和 K15，游戏失败 H17 灯亮，游戏通关 K15 灯亮。

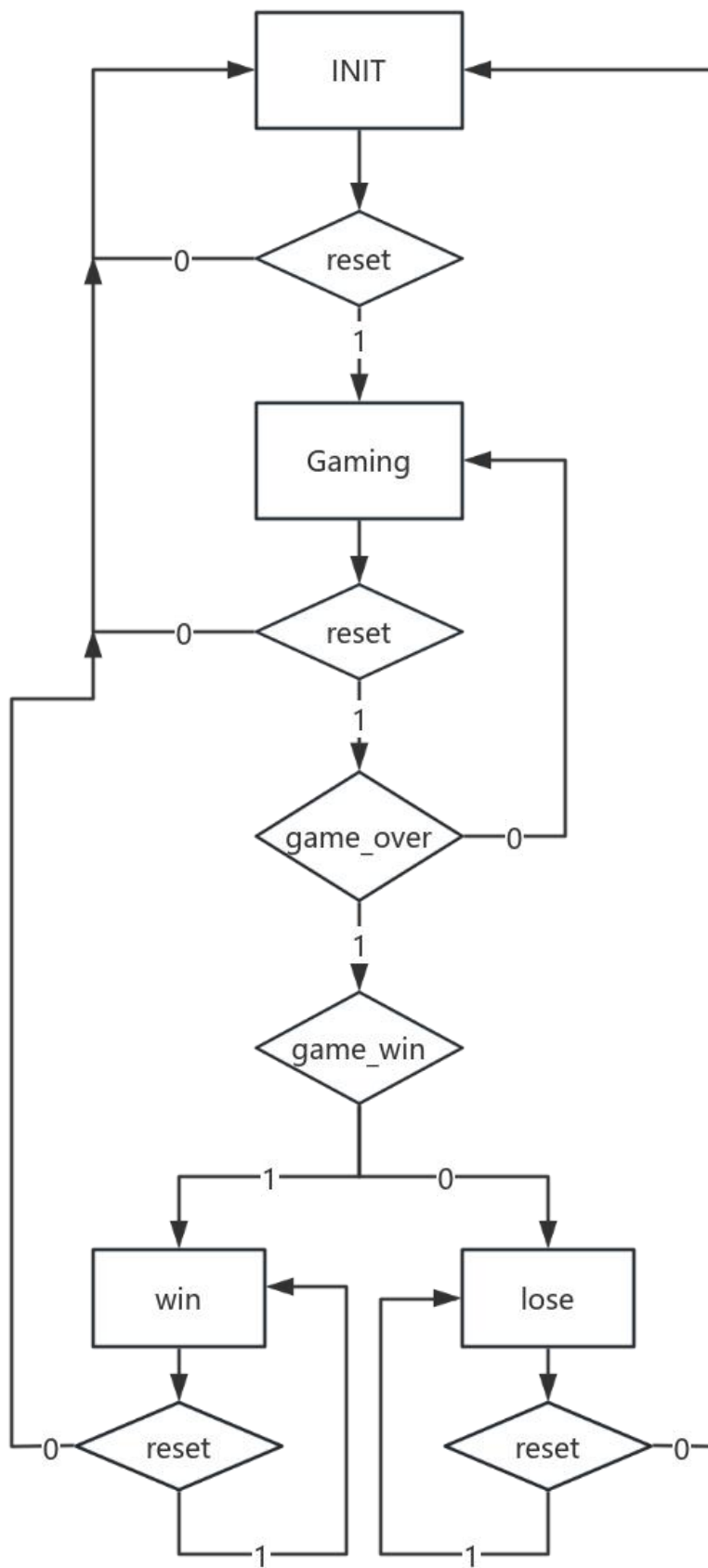
计时系统：根据 ip 核转换的 656Mhz 时钟计时，每 65000000 个时钟增加一秒，以达到计时效果，游戏时间到 160s 即通关。

输出 RGB 变化子系统：根据玩家位置、障碍物位置、游戏时长、盾牌数量等参数，改变要传入显示器的 RGB 值。

游戏参数变化子系统：根据输入子系统的输入，更改玩家的跳跃状态和技能使用状态，并根据跳跃状态改变玩家的位置(利用重力加速度模仿重力系统)；根据生成的随机数，改变障碍物出现的高度和位置；随着时间增加，加快障碍物的移动速度。

三、系统控制器设计

（要求画出所设计数字系统的 ASM 流程图，列出状态转移真值表。由状态转移真值表，求出系统控制器的次态激励函数表达式和控制命令逻辑表达式，并用 Logisim 画出系统控制器逻辑方案图。）



状态转移真值表如下：

PS			NS			
状态名	A	B	状态名	A	B	转移条件
INIT	0	0	INIT	0	0	Reset=0
INIT	0	0	Gaming	0	1	Reset=1
Gaming	0	1	INIT	0	0	Reset=0
Gaming	0	1	Gaming	0	1	Reset=1,game_over=0
Gaming	0	1	WIN	1	0	Reset=1,game_over=1 Game_win=1
Gaming	0	1	LOSE	1	1	Reset=1,game_over=1 Game_win=0
WIN	1	0	WIN	1	0	Reset=1,game_over=1 Game_win=1
WIN	1	0	INIT	0	0	Reset=0
LOSE	1	1	LOSE	1	1	Reset=1,game_over=1 Game_win=0
LOSE	1	1	INIT	0	0	Reset=0

激励信号：

$D1 = (\sim A * B + A * \sim B * game_win + B * \sim game_win) * game_over * reset;$

$D2 = (B * game_over * \sim game_win + \sim A * \sim game_win + \sim A * \sim game_over + \sim A * \sim B) * reset;$

控制命令表达式：

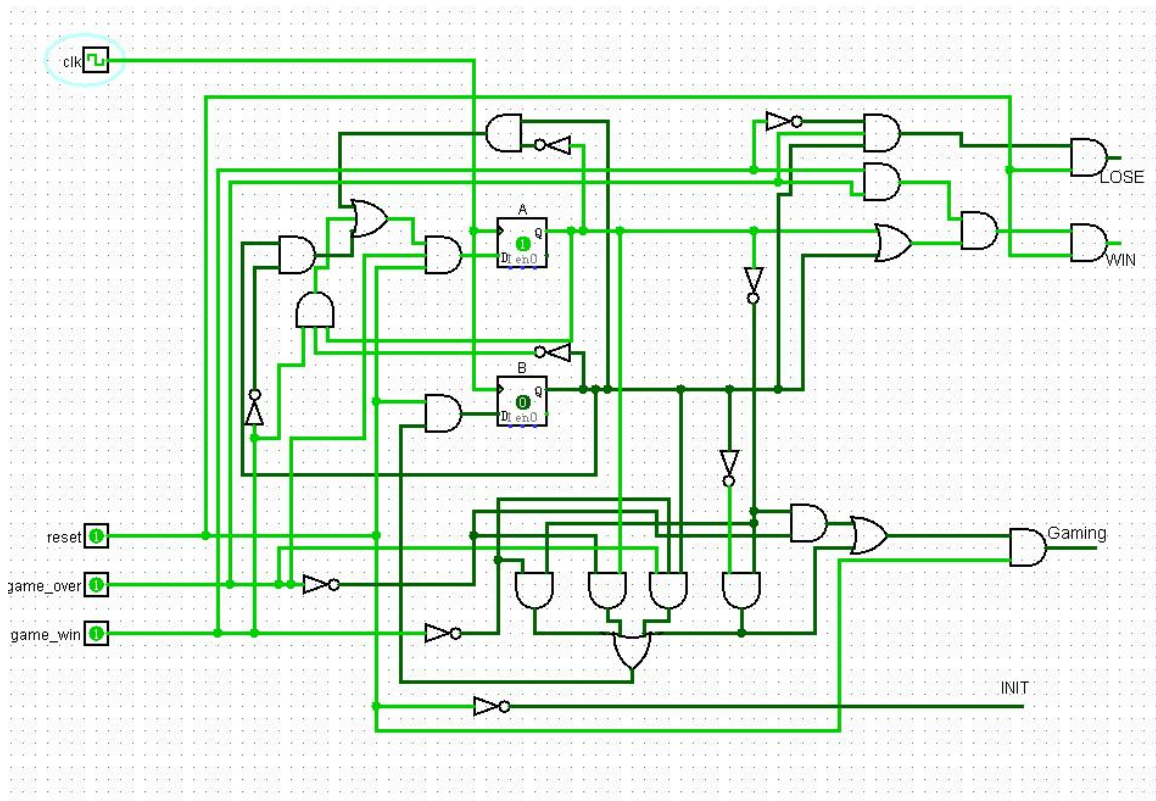
$INIT = \sim reset ;$

$Gaming = \sim A * \sim B * reset + \sim A * reset * \sim game_over;$

$WIN = (B + A) * reset * game_over * game_win;$

$LOSE = B * reset * game_over * \sim game_win;$

logicsim 原理图如下：



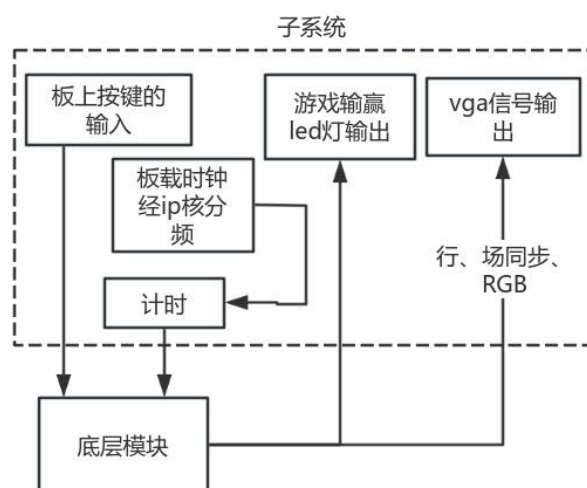
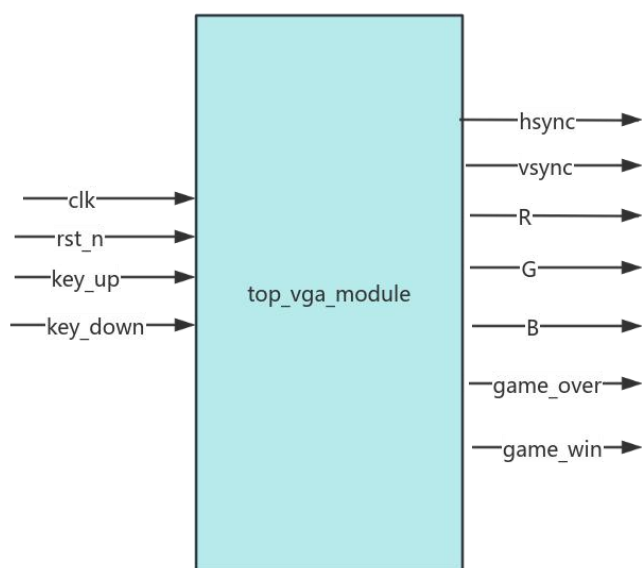
四、子系统模块建模

（该部分要求对实验中的所有子系统模块进行描述，给出各子系统的功能框图及接口信号定义，并列出各模块建模，图文描述，描述清楚设计及实现的思路，不要在此处放 verilog 代码，所有的 verilog 代码在附录部分）

①输入子系统、计时系统和输出子系统 2 和输出子系统 1 的接口部分：

我放在同一个顶层 module 中完成，即 top_vga_module 中，代码中有部分原 mp3 需要的接口，但最终实现失败，故不在框图内。

接口框图与功能框图：



各接口信号：

Clk：接板载时钟 E3

Rst_n: 重置信号，接板上开关 J15

key_up: 接 FPGA 板的按键 M18，即上键，代表跳跃信号。

key_down: 接 FPGA 板的按键 P18，即下键，代表技能信号。

Game_over: 接 FPGA 板的 led 灯 H17，亮起则游戏结束，值来自底层模块。

Game_win: 接 FPGA 板的 led 灯 K15，亮起则游戏胜利，值来自底层模块。

hsync: 行同步信号，接板载 vga 接口的 B11，值来自底层模块。

vsync: 场同步信号，接板载 vga 接口的 B12，值来自底层模块。

R[3:0]: RGB 信号红色值，接板载 vga 接口的 A4、C5、B4、A3，值来自底层模

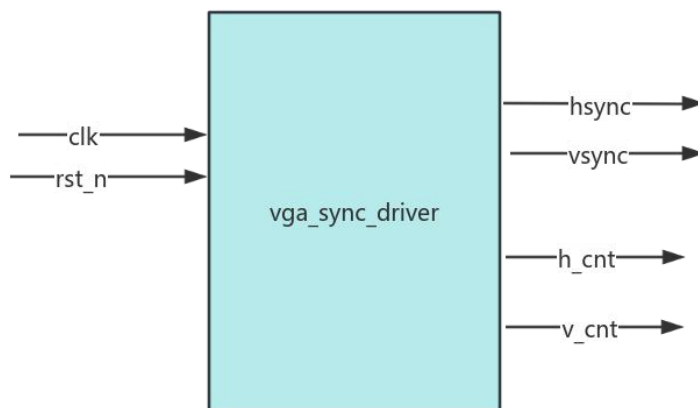
块。

G[3:0]: RGB 信号绿色值，接板载 vga 接口的 A6、B6、A5、C6，值来自底层模块。

B[3:0]: RGB 信号蓝色值，接板载 vga 接口的 D8、D7、C7、B7，值来自底层模块。

计时系统:通过板载时钟接 ip 核分频至 65mhz(显示器 1024*768 的分辨率所需的时钟频率)，每 65000000 个时钟，game_time 加 1，达到计时效果。

②输出子系统 1 的行同步、场同步驱动部分：



各接口信号及更新逻辑：

Clk：由上层模块传入

Rst_n: 重置信号，由上层模块传入

hsync: 行同步信号，有效区间为每帧的行计数 h_cnt 的 136 至 1343。

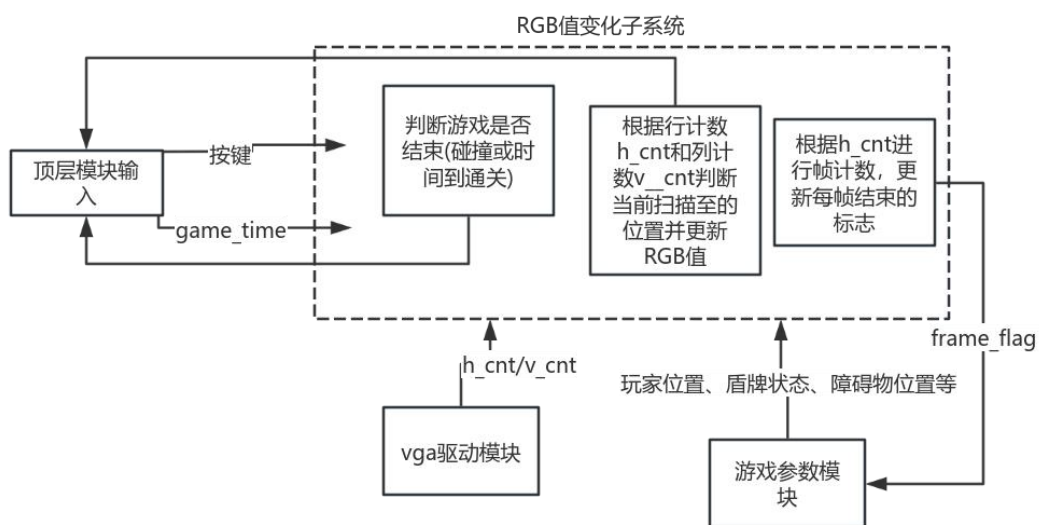
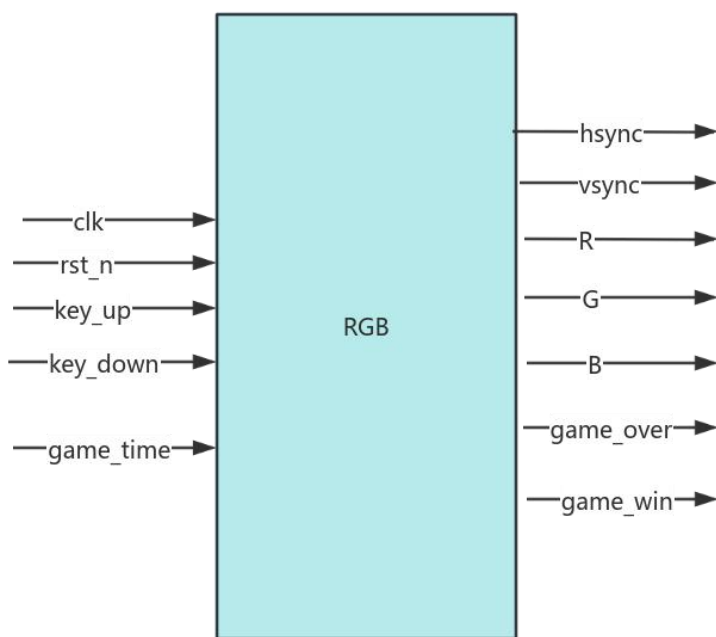
vsync: 场同步信号，有效区间为每行的列计数 v_cnt 的 6 至 805。

h_cnt:行计数，计数范围为 0-1343。

v_cnt:列计数，计数范围为 0-805。

模块功能为产生 vga 显示所需的行同步和场同步信号，根据行扫描计数器和场扫描计数器 h_cnt 和 v_cnt 的值的周期性变化，周期性变化 h_sync 和 v_sync。

③输出 RGB 变化子系统：根据玩家位置、障碍物位置、游戏时长、盾牌数量等参数，改变要传入显示器的 RGB 值。



各接口信号：

Clk：由上层模块传入

Rst_n: 重置信号，由上层模块传入

key_up: 跳跃信号，由上层模块传入。

key_down: 技能信号，由上层模块传入。

Game_time: 游戏时间信号，由上层模块传入

hsync: 行同步信号，由下层模块传入。

vsync: 场同步信号，由下层模块传入。

R[3:0]: RGB 信号红色值。

G[3:0]: RGB 信号绿色值。

B[3:0]: RGB 信号蓝色值。

Game_over:游戏结束信号, 若 collision=1(碰撞), 或 game_time>=160(游戏通关), 则 game_over 信号置 1。

Game_win: 游戏是否获胜信号, 如果是通过 game_time>=160 结束则置 1, 否则为 0。

实现设计:

其中, 是否碰撞 collision 是根据底层传入的参数进行判断, 如果盾牌未启用时, 玩家位置与障碍物位置重合则发生碰撞, 盾牌启用时则无视位置。

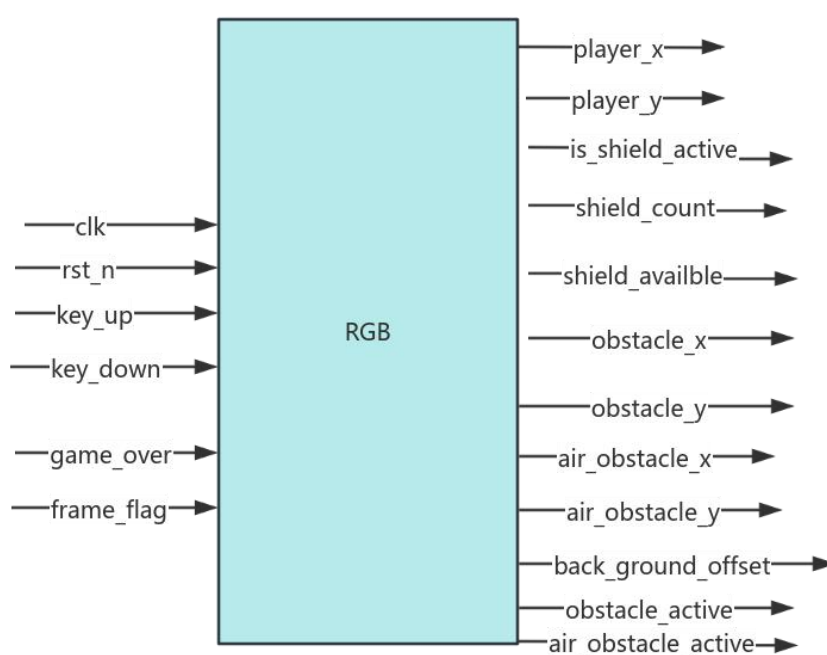
RGB 值的更改则是依赖于底层游戏参数模块的传入。根据行计数和列计数信号判断当前显示器扫描至的位置, 然后判断位置: 处于玩家的位置, 则置玩家色块的颜色(分有无盾牌两种); 处于障碍物的位置则置障碍物的颜色; 处于地面的位置则置地面的颜色。

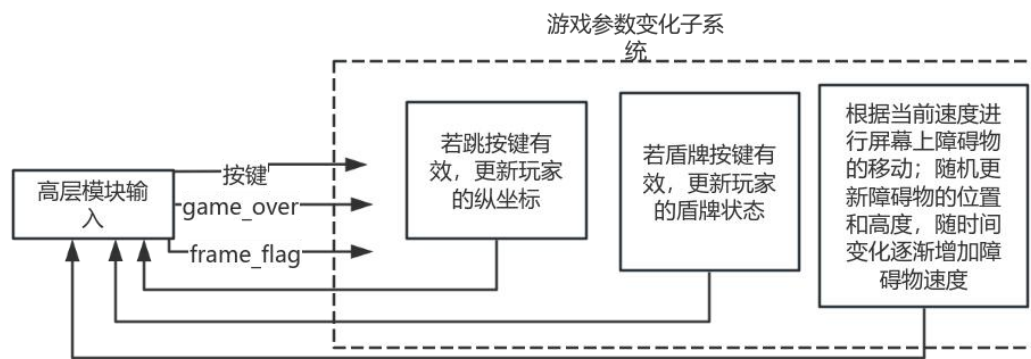
背景区域则置背景区域的颜色(为了让画面有移动的感觉, 加入竖直的淡条纹, 位置随时间变化周期性移动)。

除此之外还在顶部区域通过绘制小方块的形式显示了游戏进度、盾牌数量。Game_time 每增加十秒, 增加一个时间小方块(增加要改变 rgb 的区域)。根据当前的盾牌数量, 绘制盾牌小方块。

在当前模块中还根据 h_cnt 进行了帧结束标志 frame_flag 的更新, 并传入底层参数模块, 让参数不随时钟变化而根据帧变化, 降低了运行负担。

④游戏参数变化子系统: 更改玩家的跳跃状态和技能使用状态, 并根据跳跃状态改变玩家的位置(利用重力加速度模仿重力系统); 根据生成的随机数, 改变障碍物出现的高度和位置; 随着时间增加, 加快障碍物的移动速度。





各接口信号:

Clk: 由上层模块传入

Rst_n: 重置信号, 由上层模块传入

key_up: 跳跃信号, 由上层模块传入。

key_down: 技能信号, 由上层模块传入。

Game_over: 游戏是否结束信号, 由上层模块传入。

Frame_flag: 帧结束信号, 由上层模块传入。

Player_x: 玩家横坐标, 输出至 RGB 模块。

Player_y: 玩家纵坐标, 输出至 RGB 模块。

Is_shield_active: 盾牌有效信号, 输出至 RGB 模块。

shield_count: 盾牌使用次数, 输出至 RGB 模块。

shield_available: 盾牌是否有剩余信号, 输出至 RGB 模块。

obstacle_x: 障碍物横坐标, 输出至 RGB 模块。

obstacle_y: 障碍物纵坐标, 输出至 RGB 模块。

air_obstacle_x: 空中障碍物横坐标, 输出至 RGB 模块。

air_obstacle_y: 空中障碍物纵坐标, 输出至 RGB 模块。

background_offset: 背景条纹偏移量(位置), 输出至 RGB 模块。

obstacle_active: 障碍物是否刷新, 输出至 RGB 模块。

air_obstacle_active: 空中障碍物是否刷新, 输出至 RGB 模块。

实现设计:

玩家的位置仅能通过跳跃改变, 如果检测到跳跃按键有效, 则置玩家为“跳跃状态”, is_jumping 置 1, 在 is_jumping==1 期间, 每次(以帧为单位)均对玩家纵坐标做减 player_velocity 处理, 其中 player_velocity 初始为一个正的速度 jump_speed, 随着跳跃的进行, 不断减少, 每帧减少 GRAVITY, 以模拟重力加速度的效果, player_velocity 达到负值后玩家下落。每次玩家纵坐标变化后检测玩家是否落地, 落地则重置跳跃状态等。

盾牌系统: 如果检测到盾牌按键有效, 则置 is_shield_active 为 1, 切换为盾牌状态, 如果无效, 则置 is_shield_active 为 0。用 shield_pressed_last 来记录上一帧的按键状态避免长按多次计数。如果盾牌使用次数达到五次, 则 shield_available 置 0, 盾牌无效。

障碍物生成:根据当前随机数的值 `random_number` 与某个数取余的结果调节地面、空中障碍物的高度、左右等。

障碍物移动:根据障碍物移动的速度 `current_speed` 每帧对障碍物的横坐标进行减操作, `current_speed` 会随着游戏进行加快(上限 `MAX_SPEED`)。

五、测试模块建模

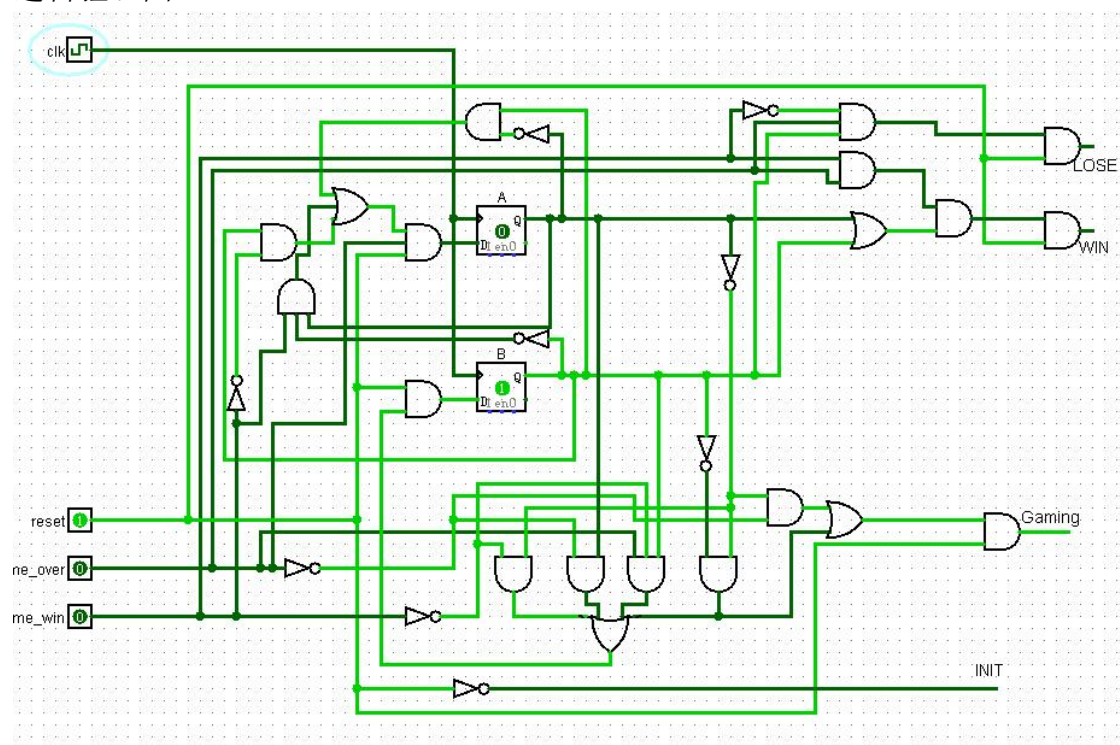
(要求列写各建模模块的 `test bench` 模块测试逻辑, 不要在此处放 `verilog` 代码, 所有的 `verilog` 代码在附录部分)

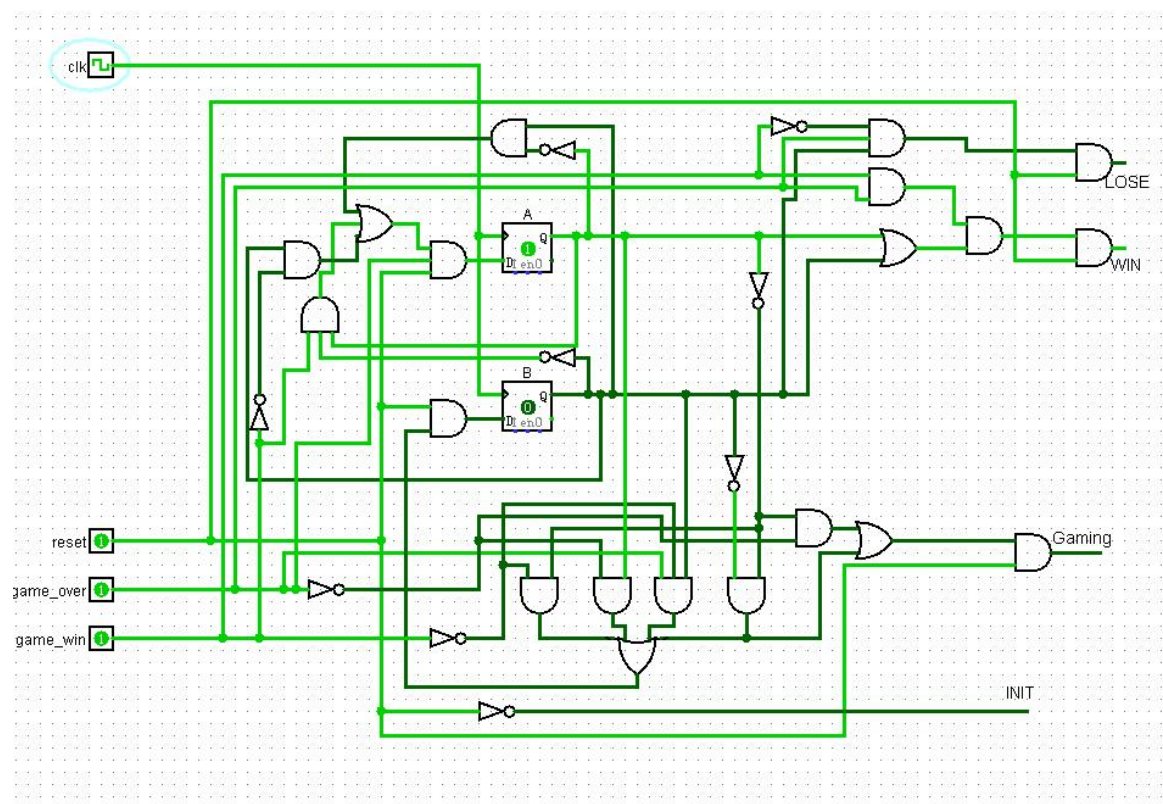
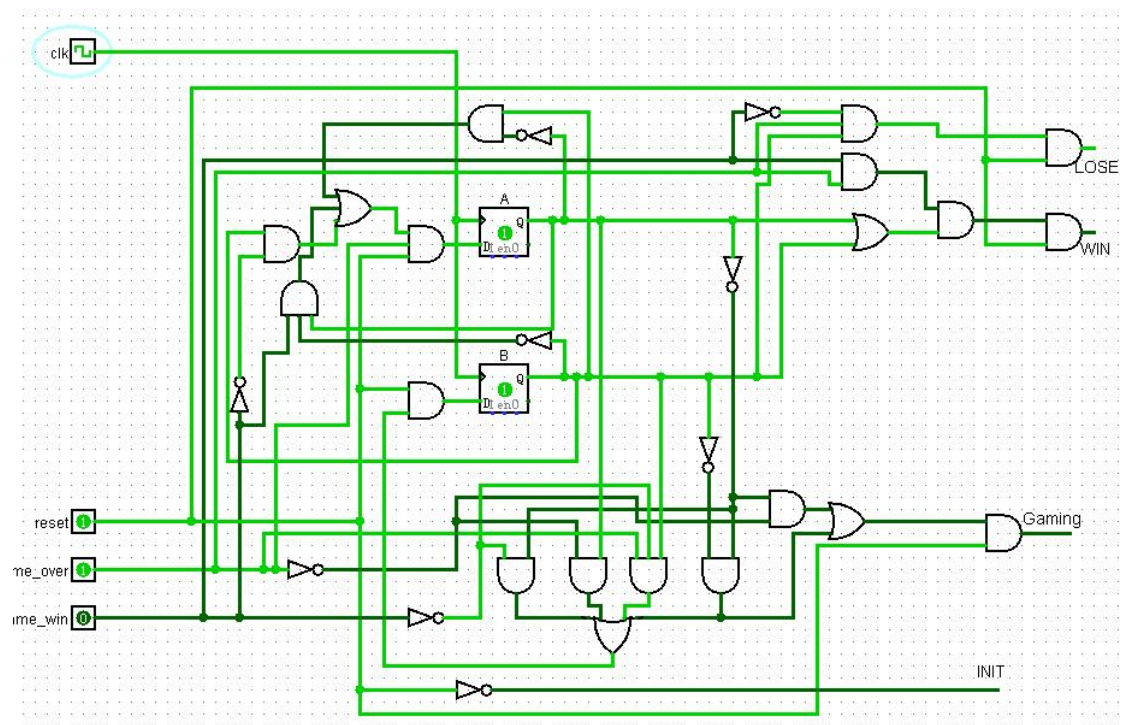
无, 通过显示屏的反馈进行模块测试。

六、实验结果

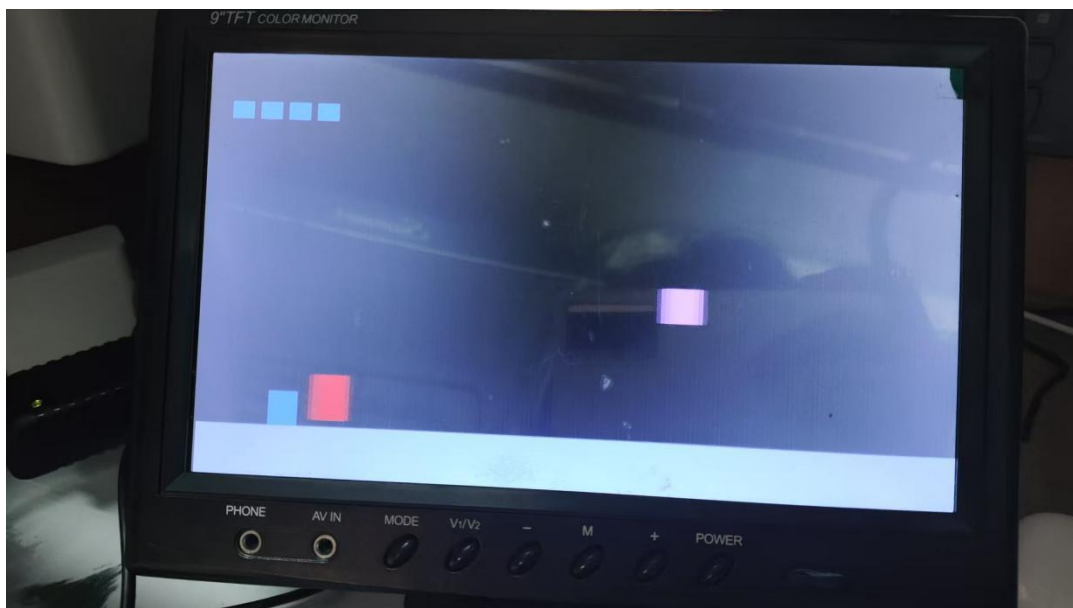
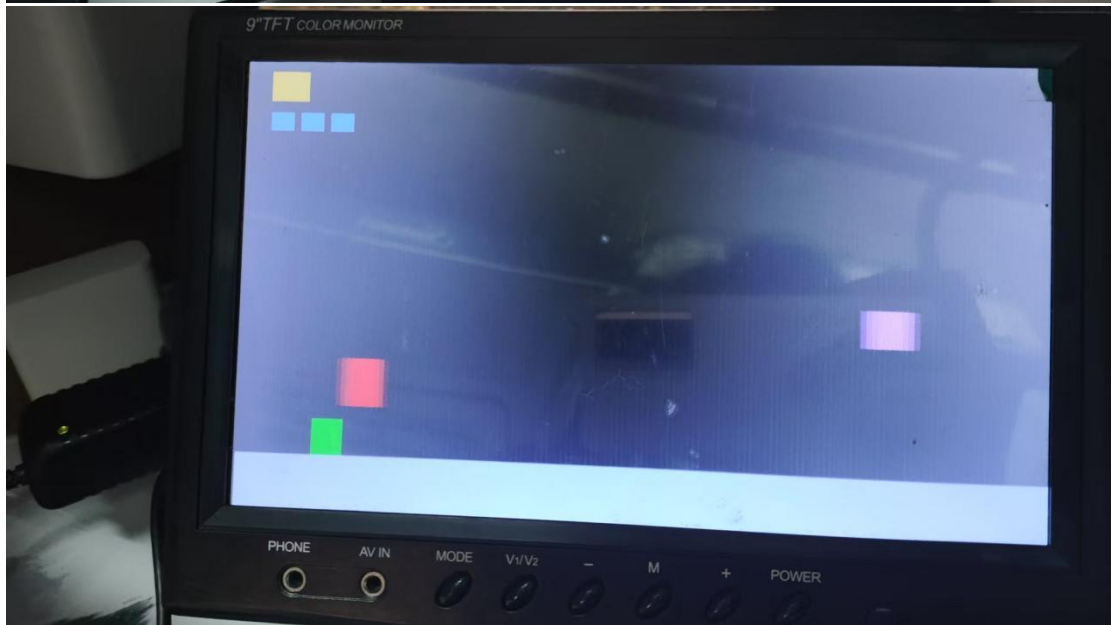
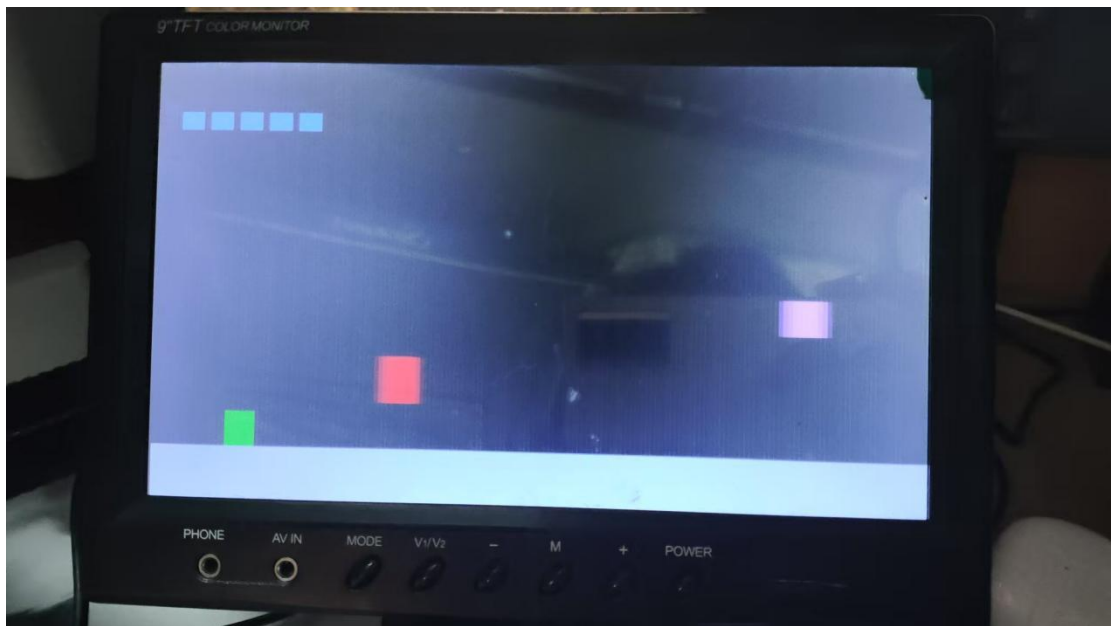
(该部分可截图说明, 可包含 `logisim` 逻辑验证图、`modelsim` 仿真波形图、以及下板后的实验结果贴图)

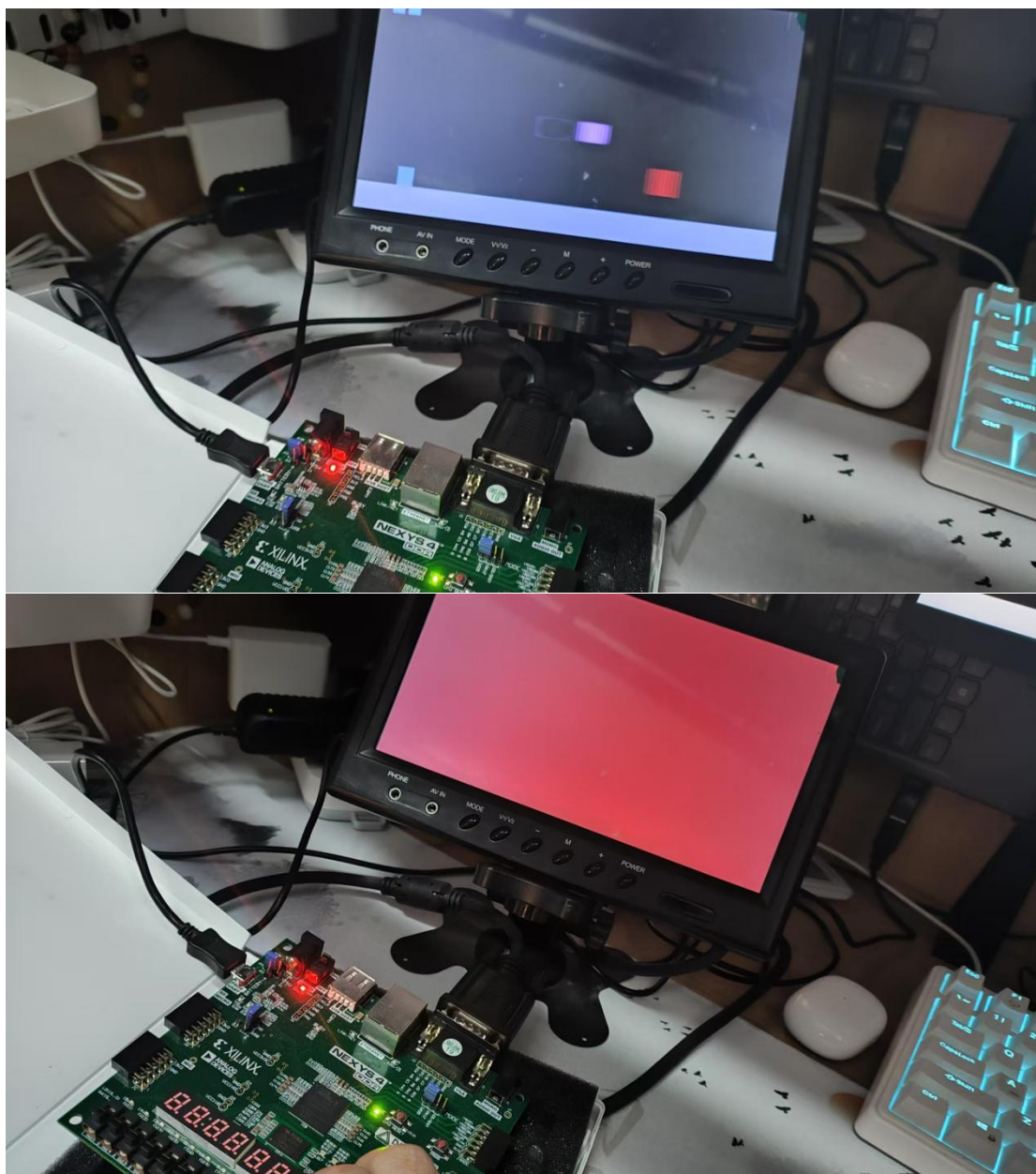
逻辑验证图:





实验结果贴图：





七、结论

本次实验中，我利用 FPGA 开发板及 VGA 显示器制作了一款模仿“chrome 小恐龙”的游戏，通过对 vga 协议的研究，我实现了行扫描、场扫描等逻辑，并根据扫描使 RGB 信号和行、场同步信号同步，达到 vga 协议的解析及显示。通过按键输入来完成游戏的控制，通过游戏参数分析和生成实现了图像的变化、游戏状态的变化，并由 RGB 改变模块改变不同位置的 RGB 从而以图像的形式显示。我还利用了 ip 核完成板载时钟的分频处理，以获取所需的时钟频率。不过遗憾的是，原先很多预想的功能没有实现，比如 mp3 模块和图片显示等。

FPGA 板是适合新手上手的可编程逻辑阵列，以其高效的性能给了使用者很好的体验，有机会我一定会再次尝试在板上开发项目。

八、心得体会及建议

项目结束，但有些遗憾的是，原先我打算在游戏过程中利用 mp3 播放背景音乐，但是经过长时间的调试，mp3 仍然没有声音，最终不得不放弃；还有图片显示也是，原先打算用提前读在 ip 核中的图片来充当玩家的图像，但最终因为一些时序逻辑的问题没有实现。这让我意识到硬件描述语言和高级语言的不同之处。在高级语言编程中，许多清晰易懂的逻辑可以通过描述轻松实现。然而，硬件设计则更加注重并行性和时序逻辑。在软件编程中，程序的执行是线性和顺序的，而在硬件设计中，许多操作是同时进行的。这种根本的差异使得即便是最简单的逻辑，在硬件中实现时也可能面临意想不到的复杂性。这一点在我的某些模块的失败中得到了充分体现。

遗憾之外，这次实验也有很多挑战和收获。在实验过程中，研究 VGA 协议的细节是一个不小的挑战。通过阅读相关资料和观察示波器信号，我逐渐掌握了如何生成所需的信号。这让我对 FPGA 的硬件描述语言（如 Verilog 或 VHDL）的使用有了更加深入的理解，同时也锻炼了我的问题解决能力，能够更快定位和修复实验中的问题。

在游戏控制方面，我通过按键输入实现了与游戏状态的交互。这一部分让我了解到用户输入如何实时影响游戏逻辑，增加了我对嵌入式系统实时性的理解。不同位置的 RGB 信号变化反映了游戏状态的变化，也让我对图像显示的基本原理有了更清晰的认识。

通过这次实验，我不仅提升了自己的硬件编程能力，还有助于我在实际项目中理解基础电子学和数字电路设计的应用。未来，我希望能进一步提高自己的 FPGA 设计能力，尝试实现更复杂的图形、音频处理功能，并探索多种输入输出接口的整合。

总的来说，这次实验让我积累了宝贵的经验，对 FPGA 技术有了更深的理解，期待在下一次的实践中做得更好！

九、附录

（该部分放 verilog 代码，包括所有设计文件和测试文件）

注：代码含未实现的功能，比如 mp3 模块的例化。

```
module top_vga_module(  
    input wire clk,           // 100M 板载时钟  
    input wire rst_n,  
    input wire key_up,        // 上键  
    input wire key_down,      // 下键  
    input wire key_left,      // 左键  
    input wire key_right,     // 右键  
    output wire hsync,  
    output wire vsync,  
    output wire [3:0] R,  
    output wire [3:0] G,  
    output wire [3:0] B,  
    output wire game_over,    // 游戏结束指示灯  
    output wire game_win,     // 添加游戏胜利指示灯  
    // mp3 相关
```

```

input wire DREQ,           //数据请求，高电平时可传输数据
output wire XCS,           // SCI 传输读写指令
output wire XDCS,          // SDI 传输数据
output wire SCK,           // 时钟
output wire SI,            // 传入 mp3
output wire XRESET         // 硬件复位，低电平有效
);

wire clk_65M;
wire locked;
reg [11:0] game_time;      // 游戏时间计数器
reg [31:0] clk_counter;    // 时钟计数器
// 计时逻辑 (基于 65MHz 时钟)
always @(posedge clk_65M or negedge rst_n) begin
    if (!rst_n) begin
        game_time <= 0;
        clk_counter <= 0;
    end
    else if (!game_over) begin
        if (clk_counter >= 65000000) begin // 1 秒
            clk_counter <= 0;
            game_time <= game_time + 1;
        end
        else begin
            clk_counter <= clk_counter + 1;
        end
    end
end

// 实例化 Clocking Wizard 模块
clk_wiz_0 inst_clk_wiz (
    .clk_in1(clk),
    .reset(~rst_n),
    .clk_out1(clk_65M),
    .locked(locked)
);

// rgb 输出模块例化
RGB instial_RGB (
    .clk(clk_65M),
    .reset_n(rst_n & locked),
    .key_up(key_up),
    .key_down(key_down),
    .key_left(key_left),
    .key_right(key_right),
    .hsync(hsync),

```

```

        .vsync(vsync),
        .R(R),
        .G(G),
        .B(B),
        .game_over(game_over),
        .game_win(game_win),
        .game_time(game_time)
    );
    // mp3
    music_driver mp3_inst(
        .clk(clk),
        .DREQ(DREQ),
        .rst(rst),
        .music_id(0),
        .XDCS(XDCS),
        .XCS(XCS),
        .XRSET(XRESET),
        .SI(SI),
        .SCK(SCK),
        .start(start)
    );
endmodule

```

```

module RGB
#(
    parameter H_CNT_BIT_WIDTH = 10'd11,
    parameter V_CNT_BIT_WIDTH = 10'd11,

    parameter H_SYNC = 10'd136,
    parameter H_BACK = 10'd160,
    parameter H_ACTIVE = 11'd1024,
    parameter H_FRONT = 10'd24,
    parameter H_TOTAL = 11'd1344,
    parameter V_SYNC = 10'd6,
    parameter V_BACK = 10'd29,
    parameter V_ACTIVE = 10'd768,
    parameter V_FRONT = 10'd3,
    parameter V_TOTAL = 10'd806,
    // 游戏参数
    parameter PLAYER_WIDTH = 40,    // 玩家宽度
    parameter PLAYER_HEIGHT = 60,   // 玩家高度
    parameter JUMP_SPEED = 20,       // 增大跳跃初始速度
    parameter GRAVITY = 1,           // 保持较小的 g
    parameter GROUND_HEIGHT = 100,  // 地面高度

```

```

parameter OBSTACLE_WIDTH = 50, // 障碍物宽度
parameter OBSTACLE_HEIGHT = 80, // 障碍物高度
parameter AIR_OBSTACLE_WIDTH = 50, // 空中障碍物宽度
parameter AIR_OBSTACLE_HEIGHT = 60, // 空中障碍物高度
parameter AIR_OBSTACLE_Y = 400, // 空中障碍物的固定高度（从上往下
数）

parameter INIT_SPEED = 5, // 初始速度
parameter MAX_SPEED = 30, // 最大速度
parameter SPEED_UP_INTERVAL = 300, // 每 300 帧增加一次速度
parameter SPEED_INCREMENT = 1, // 每次速度增加值
// 修改参数定义部分，添加时间显示相关参数
parameter TIME_BLOCK_SIZE = 50, // 增大方块大小（原来是 15）
parameter TIME_BLOCK_GAP = 10, // 调整间距（原来是 5）
parameter TIME_BLOCK_X = 30, // 保持起始 X 坐标
parameter TIME_BLOCK_Y = 20, // 保持起始 Y 坐标
parameter TIME_BLOCK_COLOR_R = 4'b1111, // 保持金色不变
parameter TIME_BLOCK_COLOR_G = 4'b1100,
parameter TIME_BLOCK_COLOR_B = 4'b0000,
// 添加盾牌显示相关参数
parameter SHIELD_BLOCK_SIZE = 30, // 盾牌方块大小
parameter SHIELD_BLOCK_GAP = 10, // 盾牌方块间距
parameter SHIELD_BLOCK_X = 30, // 起始 X 坐标
parameter SHIELD_BLOCK_Y = TIME_BLOCK_Y + TIME_BLOCK_SIZE + 20, // Y
坐标（时间方块下面）
parameter SHIELD_BLOCK_COLOR = 4'b1111 // 蓝色
)
//端口定义
(
input wire clk,
input wire reset_n,
input wire key_up,
input wire key_down,
input wire key_left,
input wire key_right,
output wire hsync,
output wire vsync,
output reg [3:0] R,
output reg [3:0] G,
output reg [3:0] B,
output reg game_over,
input wire [11:0] game_time, // 添加游戏时间输入
output reg game_win // 添加游戏胜利标志
);
reg frame_flag; //帧结束标志

```

```

reg frame_cnt=0; //帧计数器
wire [H_CNT_BIT_WIDTH-1 : 0] h_cnt;
wire [V_CNT_BIT_WIDTH-1 : 0] v_cnt;
//实例化 VGA 驱动
vga_sync_driver inst_vga_driver (
    .clk      (clk),
    .reset_n   (reset_n),
    .hsync     (hsync),
    .vsync     (vsync),
    .h_cnt     (h_cnt),
    .v_cnt     (v_cnt)
);
//帧结束标志
always @(posedge clk or negedge reset_n)
begin
    if (!reset_n) begin
        frame_flag <= 0;
    end
    else if (h_cnt == H_TOTAL - 2 && v_cnt == V_TOTAL - 1) begin // 在每
帧最后一个像素时设置标志
        frame_flag <= 1;
        if(frame_cnt < 60)
            frame_cnt <= frame_cnt + 1;
        else
            frame_cnt <= 0;
    end
    else begin
        frame_flag <= 0;
    end
end
// 在模块顶部声明这些变量
reg [4:0] block_count; // 每 10 秒一个方块
reg [10:0] rel_x;
reg [4:0] current_block;
reg [10:0] block_x;
reg [4:0] shield_block_index;
reg [10:0] rel_x_shield;
// 玩家位置
wire [H_CNT_BIT_WIDTH-1:0] player_x;
wire [V_CNT_BIT_WIDTH-1:0] player_y;
wire [H_CNT_BIT_WIDTH-1:0] obstacle_x; // 障碍物 X 坐标
wire [V_CNT_BIT_WIDTH-1:0] obstacle_y; // 障碍物 Y 坐标
wire [H_CNT_BIT_WIDTH-1:0] air_obstacle_x; // 空中障碍物 X 坐标
wire [V_CNT_BIT_WIDTH-1:0] air_obstacle_y; // 空中障碍物 Y 坐标

```

```

wire [15:0] background_offset;           // 背景偏移量
// 盾牌相关变量
wire is_shield_active;                  // 盾牌是否激活
wire [2:0] shield_count;                // 记录盾牌使用次数，最多 5 次
wire shield_available;                  // 盾牌是否还可用
// 实例化 game_para_gen 模块
game_para_gen inst_game_para_gen (
    .clk(clk),
    .reset_n(reset_n),
    .frame_flag(frame_flag),
    .key_up(key_up),
    .key_down(key_down),
    .game_over(game_over),

    .player_x(player_x),
    .player_y(player_y),
    .is_shield_active(is_shield_active),
    .shield_count(shield_count),
    .shield_available(shield_available),
    .obstacle_x(obstacle_x),
    .obstacle_y(obstacle_y),
    .air_obstacle_x(air_obstacle_x),
    .air_obstacle_y(air_obstacle_y),
    .background_offset(background_offset),
    .obstacle_active(obstacle_active),
    .air_obstacle_active(air_obstacle_active)
);
// 碰撞检测逻辑
wire collision;
// 修改碰撞检测逻辑
assign collision =
    (!is_shield_active || !shield_available) && // 没有开启盾牌或盾牌次数
    用完时才检测碰撞
    ((obstacle_active && // 地面障碍物碰撞
        (player_x + PLAYER_WIDTH > obstacle_x) &&
        (player_x < obstacle_x + OBSTACLE_WIDTH) &&
        (player_y + PLAYER_HEIGHT > obstacle_y) &&
        (player_y < obstacle_y + OBSTACLE_HEIGHT)) ||
        (air_obstacle_active && // 空中障碍物碰撞
            (player_x + PLAYER_WIDTH > air_obstacle_x) &&
            (player_x < air_obstacle_x + AIR_OBSTACLE_WIDTH) &&
            (player_y + PLAYER_HEIGHT > AIR_OBSTACLE_Y) &&
            (player_y < AIR_OBSTACLE_Y + AIR_OBSTACLE_HEIGHT)));
// 游戏结束检测

```

```

always @(posedge clk or negedge reset_n) begin
    if (!reset_n) begin
        game_over <= 0;
        game_win <= 0;
    end
    else begin
        if (collision) begin
            game_over <= 1;
            game_win <= 0;
        end
        else if (game_time >= 160) begin // 16 个方块 * 10 秒 = 160 秒
            game_over <= 1;
            game_win <= 1;
        end
    end
end

// 修改 RGB 输出逻辑
always @(posedge clk or negedge reset_n) begin
    if (!reset_n) begin
        R <= 4'b0000;
        G <= 4'b0000;
        B <= 4'b0000;
    end
    else if ((h_cnt >= H_SYNC + H_BACK) && (h_cnt < H_SYNC + H_BACK + H_ACTIVE)
&&
        (v_cnt >= V_SYNC + V_BACK) && (v_cnt < V_SYNC + V_BACK + V_ACTIVE))
begin

    // 先判断游戏状态
    if(game_over == 1 && game_win == 0) begin
        // 游戏失败 红色
        R <= 4'b1111;
        G <= 4'b0000;
        B <= 4'b0000;
    end
    else if(game_over == 1 && game_win == 1) begin
        // 游戏胜利显示绿色
        R <= 4'b0000;
        G <= 4'b1111;
        B <= 4'b0000;
    end
    else begin
        // 显示时间方块
        if ((v_cnt >= V_SYNC + V_BACK + TIME_BLOCK_Y) &&

```

```

(v_cnt < V_SYNC + V_BACK + TIME_BLOCK_Y + TIME_BLOCK_SIZE))
begin

    // 计算当前需要显示的方块数量和位置
    block_count = game_time / 10; // 每10秒一个方块
    rel_x = h_cnt - (H_SYNC + H_BACK + TIME_BLOCK_X);
    current_block = rel_x / (TIME_BLOCK_SIZE + TIME_BLOCK_GAP);
    block_x = rel_x - current_block * (TIME_BLOCK_SIZE +
TIME_BLOCK_GAP); // 修改这里

    // 修改判断条件
    if (current_block < block_count &&
        block_x < TIME_BLOCK_SIZE &&
        rel_x >= 0 &&
        rel_x < (TIME_BLOCK_SIZE + TIME_BLOCK_GAP) * 16 && //
限制总宽度

        current_block < 16) begin // 限制最大显示16个方块
        R <= TIME_BLOCK_COLOR_R;
        G <= TIME_BLOCK_COLOR_G;
        B <= TIME_BLOCK_COLOR_B;
    end
    else begin
        // 其他区域显示背景
        R <= ((h_cnt + background_offset) % 32 == 0) ? 4'b0011 :
4'b0000;
        G <= ((h_cnt + background_offset) % 32 == 0) ? 4'b0011 :
4'b0000;
        B <= ((h_cnt + background_offset) % 32 == 0) ? 4'b0011 :
4'b0000;
    end
end
// 绘制地面
else if (v_cnt >= V_SYNC + V_BACK + V_ACTIVE - GROUND_HEIGHT)
begin
    R <= 4'b1000;
    G <= 4'b1000;
    B <= 4'b1000;
end
// 绘制玩家
else if ((h_cnt >= H_SYNC + H_BACK + player_x) &&
        (h_cnt < H_SYNC + H_BACK + player_x + PLAYER_WIDTH) &&
        (v_cnt >= V_SYNC + V_BACK + player_y) &&
        (v_cnt < V_SYNC + V_BACK + player_y + PLAYER_HEIGHT))
begin

```

```

        if (is_shield_active) begin
            R <= 4'b0000;
            G <= 4'b0000;
            B <= 4'b1111; // 蓝色表示盾牌激活
        end
        else begin
            R <= 4'b0000;
            G <= 4'b1111;
            B <= 4'b0000;
        end
    end
    // 绘制障碍物
    else if (obstacle_active &&(h_cnt >= H_SYNC + H_BACK + obstacle_x)
&&
        (h_cnt < H_SYNC + H_BACK + obstacle_x + OBSTACLE_WIDTH)
&&
        (v_cnt >= V_SYNC + V_BACK + obstacle_y) &&
        (v_cnt < V_SYNC + V_BACK + obstacle_y + OBSTACLE_HEIGHT))
begin
        R <= 4'b1111;
        G <= 4'b0000;
        B <= 4'b0000;
    end
    // 绘制空中障碍物
    else if (air_obstacle_active &&(h_cnt >= H_SYNC + H_BACK +
air_obstacle_x) &&
        (h_cnt < H_SYNC + H_BACK + air_obstacle_x +
AIR_OBSTACLE_WIDTH) &&
        (v_cnt >= V_SYNC + V_BACK + AIR_OBSTACLE_Y) &&
        (v_cnt < V_SYNC + V_BACK + AIR_OBSTACLE_Y +
AIR_OBSTACLE_HEIGHT)) begin
        R <= 4'b1111;
        G <= 4'b0000;
        B <= 4'b1111; // 紫色表示空中障碍物
    end
    // 绘制剩余盾牌数
    else if ((v_cnt >= V_SYNC + V_BACK + SHIELD_BLOCK_Y) &&
        (v_cnt < V_SYNC + V_BACK + SHIELD_BLOCK_Y +
SHIELD_BLOCK_SIZE)) begin

        // 计算当前像素位置对应的方块索引

        rel_x_shield = h_cnt - (H_SYNC + H_BACK + SHIELD_BLOCK_X);

```

```

        shield_block_index = rel_x_shield / (SHIELD_BLOCK_SIZE +
SHIELD_BLOCK_GAP);

        // 如果在有效的方块范围内且是剩余的盾牌数
        if (shield_block_index < (5 - shield_count) &&
            rel_x_shield >= 0 &&
            rel_x_shield < (SHIELD_BLOCK_SIZE + SHIELD_BLOCK_GAP)
* 5 &&
            (rel_x_shield % (SHIELD_BLOCK_SIZE + SHIELD_BLOCK_GAP))
< SHIELD_BLOCK_SIZE) begin
            R <= 4'b0000;
            G <= 4'b0000;
            B <= SHIELD_BLOCK_COLOR;
        end
        else begin
            // 其他区域显示背景
            R <= ((h_cnt + background_offset) % 32 == 0) ? 4'b0011 :
4'b0000;
            G <= ((h_cnt + background_offset) % 32 == 0) ? 4'b0011 :
4'b0000;
            B <= ((h_cnt + background_offset) % 32 == 0) ? 4'b0011 :
4'b0000;
        end
    end
    // 绘制背景
    else begin
        R <= ((h_cnt + background_offset) % 32 == 0) ? 4'b0011 :
4'b0000;
        G <= ((h_cnt + background_offset) % 32 == 0) ? 4'b0011 :
4'b0000;
        B <= ((h_cnt + background_offset) % 32 == 0) ? 4'b0011 :
4'b0000;
    end
end
end
else begin
    R <= 4'b0000;
    G <= 4'b0000;
    B <= 4'b0000;
end
end
endmodule

```

```

module vga_sync_driver

```

```

#(
    parameter H_CNT_BIT_WIDTH = 10'd11,
    parameter V_CNT_BIT_WIDTH = 10'd11,

    parameter H_SYNC = 10'd136,
    parameter H_TOTAL = 11'd1344,
    parameter V_SYNC = 10'd6,
    parameter V_TOTAL = 10'd806,
    parameter HSYNC_POL = 1'b1, // HSYNC 极性, 1 为正极性
    parameter VSYNC_POL = 1'b1 // VSYNC 极性, 1 为正极性
)
//端口定义
(
    input wire clk,
    input wire reset_n,
    output reg hsync,
    output reg vsync,
    output reg [H_CNT_BIT_WIDTH-1:0] h_cnt,
    output reg [V_CNT_BIT_WIDTH-1:0] v_cnt
);
//行扫描计数器
always @ (posedge clk or negedge reset_n) begin
    if(!reset_n || h_cnt == H_TOTAL-1)
        h_cnt <= 0;
    else
        h_cnt <= h_cnt + 1;
end
//行同步信号
always @ (posedge clk or negedge reset_n) begin
    if(!reset_n)
        hsync <= ~HSYNC_POL;
    else if(h_cnt == H_SYNC-1)
        hsync <= HSYNC_POL;
    else if(h_cnt == H_TOTAL-1)
        hsync <= ~HSYNC_POL;
end
//场扫描计数器
always @ (posedge clk or negedge reset_n) begin
    if(!reset_n)
        v_cnt <= 0;
    else if(h_cnt == H_TOTAL-1 && v_cnt == V_TOTAL-1) //一帧结束
        v_cnt <= 0;
    else if(h_cnt == H_TOTAL-1)
        v_cnt <= v_cnt + 1;
end

```

```

end
//场同步信号
always @ (posedge clk or negedge reset_n) begin
    if(!reset_n)
        vsync <= ~VSYNC_POL;
    else if(h_cnt == H_TOTAL-1 && v_cnt == V_SYNC-1)
        vsync <= VSYNC_POL;
    else if(h_cnt == H_TOTAL-1 && v_cnt == V_TOTAL-1)
        vsync <= ~VSYNC_POL;
end
endmodule

```

```

module game_para_gen #(
    // 游戏参数
    parameter PLAYER_WIDTH = 40,
    parameter PLAYER_HEIGHT = 60,
    parameter JUMP_SPEED = 20,
    parameter GRAVITY = 1,
    parameter GROUND_HEIGHT = 100,
    parameter OBSTACLE_WIDTH = 50,
    parameter OBSTACLE_HEIGHT = 80,
    parameter AIR_OBSTACLE_WIDTH = 50,
    parameter AIR_OBSTACLE_HEIGHT = 60,
    parameter AIR_OBSTACLE_Y = 400,
    parameter INIT_SPEED = 5,
    parameter MAX_SPEED = 30,
    parameter SPEED_UP_INTERVAL = 300,
    parameter SPEED_INCREMENT = 1,
    parameter H_ACTIVE = 1024,
    parameter V_ACTIVE = 768
)(
    input wire clk,
    input wire reset_n,
    input wire frame_flag,
    input wire key_up,
    input wire key_down,
    input wire game_over,

    // 输出玩家相关参数
    output reg [10:0] player_x,
    output reg [10:0] player_y,
    output reg is_shield_active,
    output reg [2:0] shield_count,
    output reg shield_available,

```

```

    // 输出障碍物相关参数
    output reg [10:0] obstacle_x,
    output reg [10:0] obstacle_y,
    output reg [10:0] air_obstacle_x,
    output reg [10:0] air_obstacle_y,
    output reg [15:0] background_offset,
    output reg obstacle_active,
    output reg air_obstacle_active
);
initial shield_count=0;
reg [10:0] player_velocity;
reg is_jumping;
reg [7:0] current_speed; // 当前速度
reg [9:0] speed_counter; // 速度计数器
// 添加新的控制变量
reg [9:0] obstacle_gap; // 障碍物之间的间距
reg [9:0] obstacle_height; // 地面障碍物的高度变化
// 随机数生成器相关寄存器
reg [15:0] random_number;
reg shield_pressed_last;
// 随机数生成器 - LFSR
always @(posedge clk) begin
    if (!reset_n) begin
        random_number <= 16'hACE1;
    end
    else begin
        random_number <= {random_number[14:0],
                           random_number[15] ^ random_number[13] ^
                           random_number[12] ^ random_number[10]};
    end
end
end
// 玩家控制逻辑
always @(posedge clk or negedge reset_n) begin
    if (!reset_n) begin
        player_x <= 100;
        player_y <= V_ACTIVE - GROUND_HEIGHT - PLAYER_HEIGHT;
        player_velocity <= 0;
        is_jumping <= 0;
    end
    else if (frame_flag && !game_over) begin
        // 跳跃控制
        if (key_up && !is_jumping) begin // 按下跳跃键且不在跳跃状态
            is_jumping = 1;
            player_velocity = JUMP_SPEED;
        end
    end
end

```

```

        end
        if(is_jumping)begin
            // 更新位置
            player_y <= player_y - player_velocity;
            player_velocity <= player_velocity - GRAVITY;

            //检查是否落地
            if (player_y > V_ACTIVE - GROUND_HEIGHT - PLAYER_HEIGHT) begin
                player_y <= V_ACTIVE - GROUND_HEIGHT - PLAYER_HEIGHT;
                is_jumping <= 0;
                player_velocity <= 0;
            end
        end
    end
end
//盾牌控制逻辑
always @(posedge clk or negedge reset_n) begin
    if (!reset_n) begin
        is_shield_active <= 0;
        shield_count <= 0;
        shield_available <= 1;
        shield_pressed_last <= 0;
    end
    else if (frame_flag && !game_over) begin
        if (key_down && shield_available) begin
            // 只在按下盾牌键的第一帧计数
            if (!shield_pressed_last && shield_count < 5) begin
                shield_count <= shield_count + 1;
                if (shield_count == 4) begin // 即将达到 5 次
                    shield_available <= 0; // 用完 5 次后禁用盾牌
                end
            end
        end
        is_shield_active <= 1;
    end
    else begin
        is_shield_active <= 0;
    end
    shield_pressed_last <= key_down; // 记录这一帧的按键状态
end
end
// 障碍物移动和生成逻辑
always @(posedge clk or negedge reset_n) begin
    if (!reset_n) begin
        obstacle_x <= H_ACTIVE;
    end
end

```

```

obstacle_y <= V_ACTIVE - GROUND_HEIGHT - OBSTACLE_HEIGHT;
air_obstacle_x <= H_ACTIVE + H_ACTIVE/2;
background_offset <= 0;
current_speed <= INIT_SPEED;
speed_counter <= 0;
obstacle_active <= 1;
air_obstacle_active <= 1;
obstacle_gap <= 300; // 初始间距
obstacle_height <= OBSTACLE_HEIGHT;
end
else if (frame_flag && !game_over) begin
    // 速度控制保持不变
    if (speed_counter >= SPEED_UP_INTERVAL) begin
        speed_counter <= 0;
        if (current_speed < MAX_SPEED)
            current_speed <= current_speed + SPEED_INCREMENT;
        end
    end
    else begin
        speed_counter <= speed_counter + 1;
    end
    // 地面障碍物逻辑 - 添加随机变化
    if (obstacle_x <= current_speed) begin
        obstacle_x <= H_ACTIVE + (random_number[7:0] % 100); // 随机
        // 随机改变障碍物高度 (60-100 之间)
        obstacle_height <= OBSTACLE_HEIGHT + (random_number[15:0] %
        101);
        obstacle_y <= V_ACTIVE - GROUND_HEIGHT - (OBSTACLE_HEIGHT +
        (random_number[15:0] % 101));
    end
    else if (obstacle_active) begin
        obstacle_x <= obstacle_x - current_speed;
    end

    // 空中障碍物逻辑 - 随机高度和时机
    if (air_obstacle_x <= current_speed) begin
        air_obstacle_x <= H_ACTIVE + (random_number[7:0] % 200); // 随
        // 随机改变空中障碍物高度 (300-500 之间)
        if (air_obstacle_active)
            air_obstacle_y <= 300 + (random_number[15:4] % 201);
        end
    end
    else if (air_obstacle_active) begin
        air_obstacle_x <= air_obstacle_x - current_speed;
    end
end

```

```
        end

        // 背景移动逻辑保持不变
        background_offset <= background_offset + current_speed;
    end
end
endmodule
```